AgentWorkShop: A Configurable Framework for Multi-Scenario Industrial Agent Integration

Anonymous Authors

***Abstract*—Integrating industrial agents with production context and heterogeneous equipment while preserving process-specific constraints remains challenging. We present AgentWorkShop, a modular framework that combines agent collaboration and configurable execution harnesses with line–product–recipe–run context, analysis and modeling services, semantic data-acquisition and write-control nodes, and protocol drivers. Scene blueprints define signal maps, objectives, and guards; channel profiles, tool policies, and per-agent bindings scope available operations. Agents or callers invoke named node-exploration or recipe workflows, whose route-specific checks and records distinguish command confirmation from process observation. Evaluation found that a historical seed-42 campaign met terminal checks in four configured simulator scenes. Separate protocol tests covered five acquisition families and four writable mappings, and nine deployment-suite runs passed 782 deterministic assertions. A profile and composition suite passed 65 checks using temporary storage and mock I/O, rather than process-level cross-scene trials. Live-engine and controller archives and one cadence-overridden recipe trajectory exercise complementary workflows; the recipe case saved but did not apply its accepted version. These findings provide evidence that AgentWorkShop offers a configurable integration path for the tested simulator and software-test workflows. Matched current-version cross-scene trials are still needed to validate profile and binding adaptation; the evidence also does not establish multi-harness portability, physical transfer, or comparative control benefit.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

LARGE language-model agents are being explored for plant-facing tasks. MADE generates control sequences for operator review [1]; Köhle et al. organize agents around modular factory skills [2]; and Waseem et al. study agent-based production scheduling [3]. These studies demonstrate useful agent capabilities under different task and integration assumptions. Across plants, however, agent roles, signals, protocols, objectives, and production context vary. The selected studies do not jointly evaluate a configurable framework that connects agent collaboration, scene-specific production context, and heterogeneous process I/O across these differences.

Consider injection molding. An agent team may use recent measurements and process knowledge to diagnose insufficient packing and propose a holding- pressure change. To carry that proposal into an auditable production task, a system must resolve the active line, product, recipe, and run; map the quantity to the correct device interface; check that the requested operation is available for that context; and retain the command evidence alongside subsequent quality observations. Another process may use different signals, protocols, or objectives while reusing the same collaboration and runtime services. The design problem is therefore to make scene-specific variation configurable while keeping each operation tied to its production context.

This paper presents AgentWorkShop, a configurable industrial-agent framework that addresses this integration problem. Its architecture connects collaborative tasks and execution-harness adapters to production context, analysis and modeling services, semantic data-acquisition and write-control nodes, and protocol-specific drivers. Scene blueprints define signal maps, objectives, and guards; channel profiles, tool policies, and per-agent bindings scope the operations available to each task. An agent or caller invokes a named operation, after which shared runtime checks validate and dispatch the request. The framework does not autonomously select an execution route. It keeps the proposal, command confirmation, and process observation as distinct records. Figure 1 presents the system architecture, and Fig. 2 shows the cross-module task flow. Section IV details the node-exploration and recipe lifecycles.

The paper makes three contributions:

1) It derives five integration requirements and six design principles for connecting agent collaboration, production context, process I/O, and route-specific evidence, and uses them to organize the system design.
2) It specifies AgentWorkShop's cross-module execution flow and software modules, including task and harness integration, production-context configuration, analysis and modeling services, semantic device access, and route-specific evidence handling.
3) It reports complementary evidence from a historical four-scene simulator campaign, protocol tests, current profile/composition checks with mock I/O, live-engine and controller runs, a deployment suite, and a recipe-path case. These archives exercise different components and workflows; they do not establish current-version cross-scene profile/binding adaptation, multi-harness portability, or a paired strategy comparison.

Section II reviews related industrial-agent and integration work. Section III presents the design rationale, architecture, and cross-module execution flow; Section IV explains module responsibilities and their integrated operation. Section V describes the experiments and results, and Sections VI and VII discuss the evidence limits and conclude the paper.

II. Related Work

This section connects AgentWorkShop to two complementary bodies of work: plant-facing industrial agents, and the

TABLE I
SELECTED PLANT-FACING AGENT SYSTEMS AND THEIR INTEGRATION SETTINGS. EVALUATION EVIDENCE IS REPORTED WITHIN EACH SOURCE'S SCOPE AND IS NOT A DIRECT PERFORMANCE COMPARISON.

System	Application scope	Industrial integration	Source-reported evaluation
MADE [1]	Operator-reviewed control-sequence generation	Structured OPC Unified Architecture (OPC UA) commands and authorization	40 control tasks; virtual and physical demonstrations
Köhle et al. [2]	Module agents and factory skills	OPC UA and Message Queuing Telemetry Transport (MQTT) with component-state tool availability	Three agent architectures across nine production challenges, including silent-fault recovery
Waseem et al. [3]	LLM-based robot-assignment scheduling	Prompt-constrained controller for a simulated line	Scheduling throughput
AgentWorkShop (this work)	Collaborative tasks, configurable agent harnesses, and analysis/model services	Production context and semantic process nodes over multiple protocol mappings	Historical four-scene simulator archive; separately archived protocol, software-suite, and recipe-route evidence

production and automation abstractions that define context, tool access, and operational constraints. Table I compares selected agent systems; the research gap is limited to these comparators rather than attributed to the wider literature.

A. Industrial Agents and Plant Integration

Industrial-agent systems span supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Biskupovic et al. study agent-based supervision for service-oriented industrial systems [4]; Agents4PLC and InstructMPC address PLC code generation and human–LLM context-aware control, respectively [5], [6]. Manufacturing studies consider dynamic resource-capability exploration [7], multi-agent shop-floor coordination [8], and digital-twin-supported fault handling [9]. Natural-language machine access [10] and real-time LLM control of a robot-operated serial line [3] extend this work to operator interfaces and robotic-line control. Together, these studies address different operational contexts, interfaces, and allocations of responsibility between agents, operators, and controllers. Accordingly, the table compares three representative integration settings: reviewed control-sequence generation, module-level manufacturing skills, and line scheduling. It reports each source's evidence within its own scope and is not a direct performance comparison.

The comparison highlights distinct design emphases rather than a ranking. AgentWorkShop treats authority and command confirmation as operational services, consistent with safety-aware decision support [11]; it does not replace process protection or establish operator effectiveness.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) standardizes tool discovery and calls with request-scoped authorization [12], whereas ISA-88 and ISA-95 describe recipes and production operations [13], [14]. Run-to-run methods connect process observations to successive production runs [15]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [16], [17]. Together, these works provide complementary abstractions for production context, tool interaction, and information flow. They do not, by themselves, specify the shared integration architecture presented here.

Industrial security and record requirements further constrain that architecture: FDA guidance discusses trustworthy records and audit trails [18], IEC 62443-3-3 specifies system-security requirements [19], and DNP3 includes select-before-operate [20]. Control-assurance methods address state evolution through shielding, control-barrier functions, or Simplex architectures [21]–[23]. These standards and methods inform the boundaries within which an industrial-agent framework must operate; they are not contributions claimed by AgentWorkShop.

Within the three representative systems in Table I, the evaluated settings do not assess one configurable composition of agent harnesses, production context, semantic process I/O, and distinct node and recipe workflows across scenes. This bounded gap motivates AgentWorkShop as an integration architecture and implementation. The experiments evaluate complementary parts of that implementation, while current profile and per-agent-binding adaptation across scenes remains unvalidated.

III. SYSTEM DESIGN

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. A useful framework must let these differences be configured without changing the shared task runtime, while ensuring that a proposed operation is checked against the active production context before it reaches a device. It must also retain enough route-specific evidence to distinguish a returned command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements follow from this integration problem. Scene variability requires signal maps, objectives, and guards to be represented as configuration. Harness variability calls for registered adapters behind a shared runtime interface. Authority requires a fresh execution-time check, with explicit approval for manual bindings. Observability requires command confirmation and process response to remain separate records. Extensibility requires new engines, tools, and protocol drivers to be registered without changing the common task path.

These requirements lead to six design rules: represent scene-specific differences as versioned data; extend the runtime

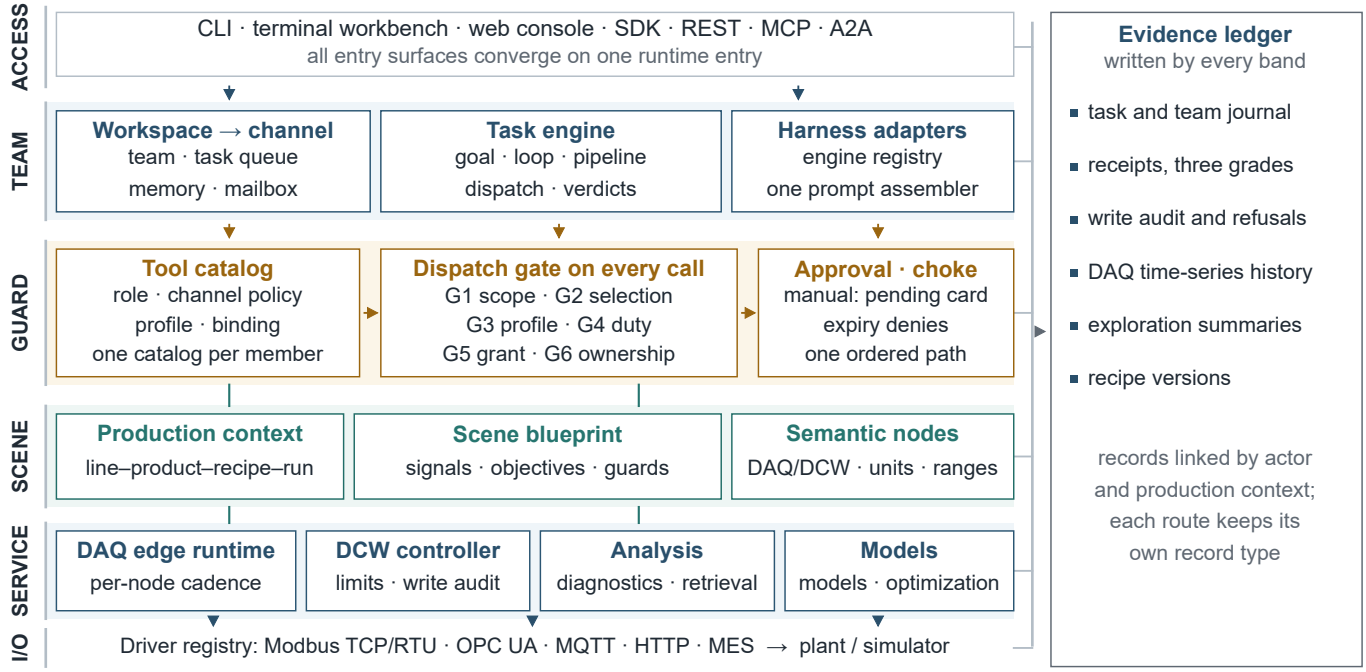


Fig. 1. AgentWorkShop software architecture. Access, collaboration, governance, production context, services, and field I/O form six connected bands. Registered harness and driver interfaces expose extension points; the evidence ledger retains route-specific records across the bands.

through registered interfaces; keep an agent's proposal separate from execution; treat the tool catalog as a view rather than an authorization boundary; route actuator writes through a common admission point; and retain evidence according to the operation that produced it. The framework does not choose an agent's strategy or replace controller and safety functions. It supplies the integration path through which a named operation is contextualized, authorized, dispatched, and recorded.

B. Layered Architecture

Fig. 1 maps the reference implementation onto six bands and a shared evidence ledger. The access band exposes command-line, web, SDK, REST, MCP, and A2A entry surfaces. The collaboration band contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. The governance band projects an operation catalog and checks each invocation. The context band binds a scene blueprint to the active line–product–recipe–run tuple and its semantic DAQ/DCW nodes. The service band provides acquisition, write control, analysis, retrieval, and optional model services. The field-I/O band resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility. Entry surfaces converge on the same task runtime; a harness adapter supplies the agent-specific interaction; semantic nodes hide protocol details from the task; and the driver registry provides the device-facing extension point. The evidence ledger records task, command, observation, and version events against their actor and production context. It does not collapse them into a single success flag.

C. Cross-Module Execution Flow

Fig. 2 summarizes a task turn from entry to returned evidence. The runtime resolves the production context and assembles the agent's prompt before invoking a registered harness. A named operation then passes through catalog projection and execution-time dispatch checks. Its operation family selects a DAQ/MES or analysis service, or a DCW/recipe handler; actuator writes additionally pass approval when required and the actuator admission checks. The resulting receipt and any later observation are retained as separate evidence and returned to the task runtime.

Algorithm 3 expresses this interaction as a cross-module reference flow. It describes the contracts between registered components, not a single orchestrator function. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

The architecture in Fig. 1 is implemented as cooperating modules rather than a single monolithic agent. Shared interfaces connect task execution to configuration, industrial services, protocol drivers, and route-specific evidence. This section describes those responsibilities and the checks that preserve their boundaries.

A. Task Runtime and Harness Adapters

An access surface creates or joins a workspace channel, where the task engine manages members, task queues, shared memory, and execution templates. The harness registry instantiates an adapter behind a common agent interface; the prompt builder combines the task with the active scene brief, industrial context, channel profile, agent binding, and retrieved memory. Tool calls return to the shared dispatcher regardless

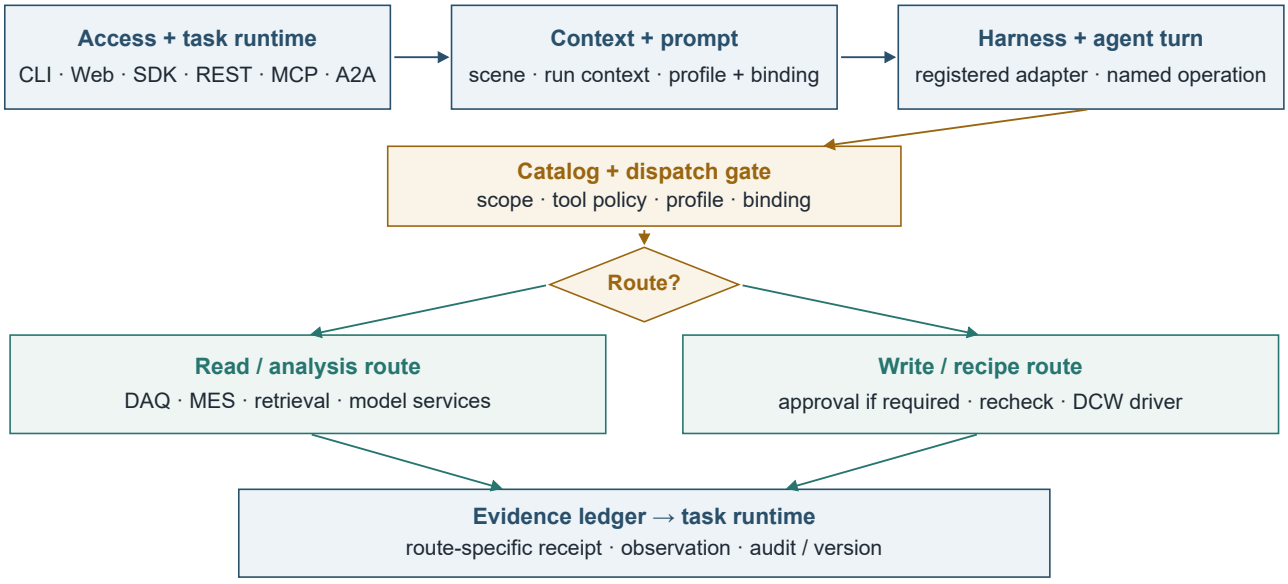


Fig. 2. Cross-module task execution in AgentWorkShop. Entry and task services resolve context and invoke a registered harness; each named operation is checked before routing to read/analysis or write/recipe services. Composite operations may invoke both service and device modules; refused calls return without field I/O. Route results pass through the evidence ledger to the task runtime. Route-specific checks are detailed in Section IV.

Require: task q, channel c, bound agent a
Ensure: task result and route-linked evidence
$x \leftarrow \textsc{ResolveContext}(c, q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x, c.profile, a.binding, Memory(q))$
$t \leftarrow \textsc{RunRegisteredHarness}(a, p)$
for all named operation u emitted by t **do**
  $C \leftarrow \textsc{ProjectCatalog}(a.role, c.profile, c.policy, a.binding)$
  if not DispatchGate(u,C,x) **then**
    RecordRefusal(u,x); **continue**
  end if
  if u requires manual approval **then**
    $d \leftarrow \textsc{AwaitApproval}(u)$
    if $d \neq approved$ **or not** Revalidate(u,x) **then**
      RecordRefusal(u,x); **continue**
    end if
  end if
  $r \leftarrow \textsc{RouteToHandler}(u)$
  if u actuates a device **then**
    $r \leftarrow \textsc{AdmitAndDispatch}(u, r, x)$
  end if
  $e \leftarrow \textsc{GradeAndCollectEvidence}(u, r, x)$
  RecordAndReturn(e,t)
end for
return task state and retained evidence

Fig. 3. Cross-module task execution in AgentWorkShop.

of which registered harness produced them. The registry is an extension point; the experiments do not establish portability across harnesses.

B. Production Context and Configuration

Scene blueprints describe devices, signal maps, objectives, and process guards. A task resolves these definitions against its active line–product–recipe–run context. The channel profile selects the available capability family, the tool policy narrows the operation catalog, and the agent binding scopes the nodes, controls, and recipe operations available to that member. These filters can restrict one another but cannot broaden the underlying device range. The dispatcher re-reads policy at execution, so a cached catalog does not retain authority after a grant or policy change.

C. Acquisition, Analysis, and Modeling Services

Semantic DAQ nodes expose typed values, engineering units, ranges, and sampling settings while hiding protocol-specific reads. A per-node DAQ runtime schedules acquisition and stores timestamped observations; the gateway aggregates node pipelines. MES-facing operations provide the configured production-data path. Analysis and retrieval services consume these records, while optional model, training, optimization, and digital-twin services are invoked through their registered operation handlers. These services complement the agent workflow; AgentWorkShop does not prescribe a single optimization strategy.

D. Semantic I/O and Actuation

The driver registry resolves a semantic node to a built-in or plugin driver. DAQ handlers use the node's acquisition interface; DCW handlers route a write or readback through the controller and the configured protocol driver. This separation lets recipes and agents refer to semantic controls without

embedding Modbus, OPC UA, MQTT, HTTP, or MES transport details in their operation arguments. The supported driver mappings are implementation interfaces; only the mappings and workflows enumerated in Section V are experimentally exercised.

E. Shared Dispatch and Write Admission

Every named operation passes the dispatcher. Its ordered checks validate invocation scope, channel tool selection, profile, duty separation, the agent's binding and runtime grant, and ownership of referenced resources. A refusal returns the excluding condition and is recorded. A catalog entry therefore describes availability but does not authorize execution. Operations under a manual binding suspend for approval; after approval, the executor revalidates authority and active limits before dispatch.

All actuator writes converge on the DCW admission point. For an online node write, the admissible interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{1}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits. Missing optional limits impose no restriction, whereas an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch applies the hard, baseline, and product limits to the replacement recipe rather than intersecting with the previous recipe window. The actuator path also enforces step, interval, and hold-window checks. These admission rules constrain commands; process objectives and coupled variable guards are assessed from later observations. The framework does not replace independent PLC or safety-instrumented protection.

F. Integrated Operation and Route-Specific Evidence

A recipe trial illustrates how the modules cooperate. The task runtime delivers a named `recipe_trial` call from the agent harness; the dispatcher checks the agent's recipe binding and execution scope; the recipe handler checks the active run, candidate parameters, and route timing; and a manual binding requests approval before the DCW controller sends the batch through its driver. The handler returns a per-parameter receipt. The caller then obtains DAQ observations and can request a separate update, apply, or rollback operation. A trial receipt confirms command handling; it does not by itself establish process attainment or persist a new recipe version.

The routes retain different records because they answer different questions. A driver acknowledgment reports command evidence; DAQ reports a process observation; an exploration summary records a bounded probe; and a recipe version records a saved definition. Their relation through actor and production context allows an execution to be inspected without treating these outcomes as interchangeable.

G. Route-Specific Lifecycles and Evidence

a) Timing and approval: Timing guards are route-specific. Online node writes enforce an effective minimum spacing of 60 s. Recipe dispatch uses a per-recipe interval anchored to approval, while trials also use a configurable per-line cadence (default 300 s). The recipe interval can refuse a proposal before an approval request is created. Checks occur at proposal or execution according to the operation, and experiment-specific pacing overrides are reported with the corresponding results.

Manual bindings create an approval request containing the actor, node or recipe, candidate values, applicable limits, expiry, and parameter bases. The default wait expires after 180 s and refuses execution. After approval, the executor rechecks authorization and active limits to account for context changes during the wait. For `recipe_apply`, a changed definition requires a new proposal; an unchanged approval dispatches a copy of the approved parameter snapshot. A line-level recipe mode defaults to manual; changing it to automatic requires administrator confirmation. The approval record authorizes a workflow action and does not replace process protection.

b) Node probe: The bounded node probe is exposed as `optimization_explore` to the Auto Modeling Lab (AML) optimization profile (`aml_optimization`). The profile validates a bound control and target DAQ scope, captures a starting control and target baseline, invokes the internal DCW write path, then waits 10–300 s (default 45 s) and queries a trailing 45-s target window before saving an exploration summary. Since each window is relative to its query time, approval can separate baseline sampling from actuation; a 10-s wait can leave about 35 s of pre-write samples in the later window. The command receipt and target summary are therefore returned separately. A successful changed-value write opens a node-optimization record; refusal or expired approval creates a rejected summary without a post-write sample.

c) Recipe operations: `recipe_trial` checks the bound recipe and active run, then dispatches candidate overrides as a batch with per-parameter verification. The caller subsequently queries DAQ and decides whether to save a new definition with `recipe_update`, restore one with `recipe_rollback`, or dispatch a version with `recipe_apply`. These steps are separate: a trial does not save or apply a version, and endpoint verification is not a process-response measurement. Public direct-node control is closed; the bounded probe and recipe operations are the configured operation surfaces. The historical four-scene results use an earlier direct-node path and do not test current profile or binding adaptation across scenes.

d) Command verification: Command confirmation and process response answer different questions. A driver ACK is graded as `readback-verified`, `transport-ack`, or `unverified`. A matching endpoint read supports the first grade; a transport or MES application receipt supports the second. Recipe batch verification assigns `verified`, `unverified`, or `failed`; if no matching readback is available, it attempts up to three independent reads, 2 s apart. A matching retry upgrades the final ACK, while exhausted retries remain unverified. The initial ACK remains in write history and the final ACK and verdict are retained with the batch. Per-parameter receipts preserve successful outcomes when another

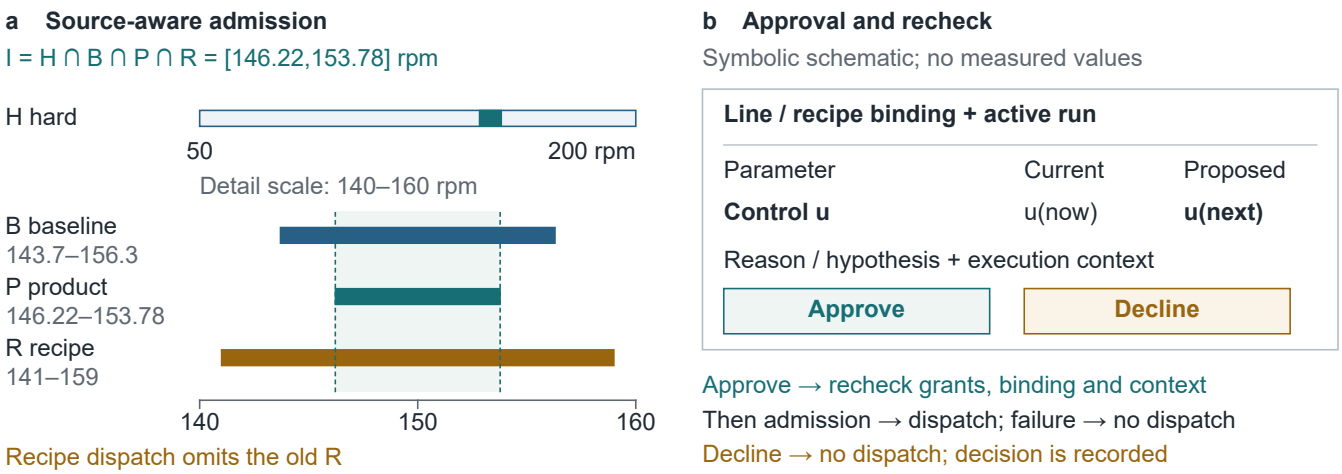


Fig. 4. Admission and approval boundaries. (a) Online node interventions use the four-layer value interval of Eq. (1); recipe dispatch omits the prior recipe window. (b) Human approval binds a proposed action and authority fields; execution checks run again after approval, which does not reserve equipment state.

parameter fails. A readback-verified setpoint does not establish target attainment.

e) Observation, verdict, and compensation: The caller performs recipe-path observation after batch dispatch. The October recipe case therefore compares a recipe definition, endpoint readback, and DAQ sample as separate representations. Saving a definition, applying it to equipment, and observing its process response are separately evidenced operations. The evaluation does not combine these route-specific records into one universal end-to-end record.

An open node record provides a monitoring backstop. After 120 s it polls every 30 s; three cumulative samples outside the recipe monitoring window trigger a system-attributed rollback request toward the preceding value. Missing samples do not count as breaches. A caller's `keep`, `rollback`, or `uncertain` verdict records its interpretation; compensation is a new command subject to its applicable admission policy. The write audit deduplicates the same value on the same node within 10 s, while batch receipts, exploration summaries, DAQ history, and recipe versions retain their distinct lifecycles. Linking these records by actor and production context allows the command, endpoint evidence, and process observation to be inspected without treating them as interchangeable.

V. Experimental Evaluation

The evaluation is organized into six evidence groups, not six research questions. Q4 first presents the historical four-scene campaign to establish the process settings; Q1 then tests acquisition and policy-checked writes over multiple protocol mappings. Q2 records two live-engine tasks under one harness, provider, and model setup, while Q3 follows one deterministic controller over three seeds. Q5 assesses deployment-suite behavior, and Q6 examines a recipe-path lifecycle. All process experiments use repository simulators. These groups are complementary archives, not one joint current-version trial or a paired strategy comparison; their source mappings are given in the reproducibility notes.

A. Evidence Units and Experimental Procedure

Q1–Q4 use the seed-42 integrated archive (R1) and its raw scenario records; the admission ablation uses a separate companion run (R2). R1 can be audited, but its historical runtime worktree is not archived, so the recorded source commit does not guarantee exact replay. These historical missions use the direct-node interface and scripted proposal policies, apart from the two Q2 live-engine tasks. Q3 additionally uses seeds 43–44; Q5 uses nine deployment-suite executions, and Q6 uses the October 18 recipe archive. The cast-film reference preset uses an author-defined objective J combining thickness conformance, defects, energy, and throughput.

Separately, 65 channel/profile and composition checks passed in an October 20 archive and an October 24 rerun with temporary storage and mock I/O. They cover approval outcomes, starting-control capture, and approved-recipe snapshot consistency; they are not process-level cross-scene trials. For each Q4 mission, the archive retains the configured scene, initial controls, proposed writes, available command evidence, post-wait DAQ observations, and terminal guard result. Each process contributes one seed-42 trajectory; writes within a trajectory are sequential observations, not independent trials.

The deployment regression spans 20 phases, from commissioning and protocol I/O to agent missions, controller reuse, and concurrent processes. Its author-defined checks report rejected probes, per-parameter confirmation, recipe/readback/mirror agreement, process-target margins, and event spans. The nine sequential runs and 782 deterministic assertions are summarized in Table IV; assertions are software checks, not plant samples. Because each protocol line has six writes, latency is reported as the archived upper median and observed maximum, not as a tail percentile.

B. Historical Four-Process Simulator Missions (Q4)

The concurrent campaign provides one seed-42 trajectory per process. Figure 5 pairs archived manipulated controls u with the corresponding response y rather than showing response alone. All four terminal attainment checks passed; two lay near a target boundary.

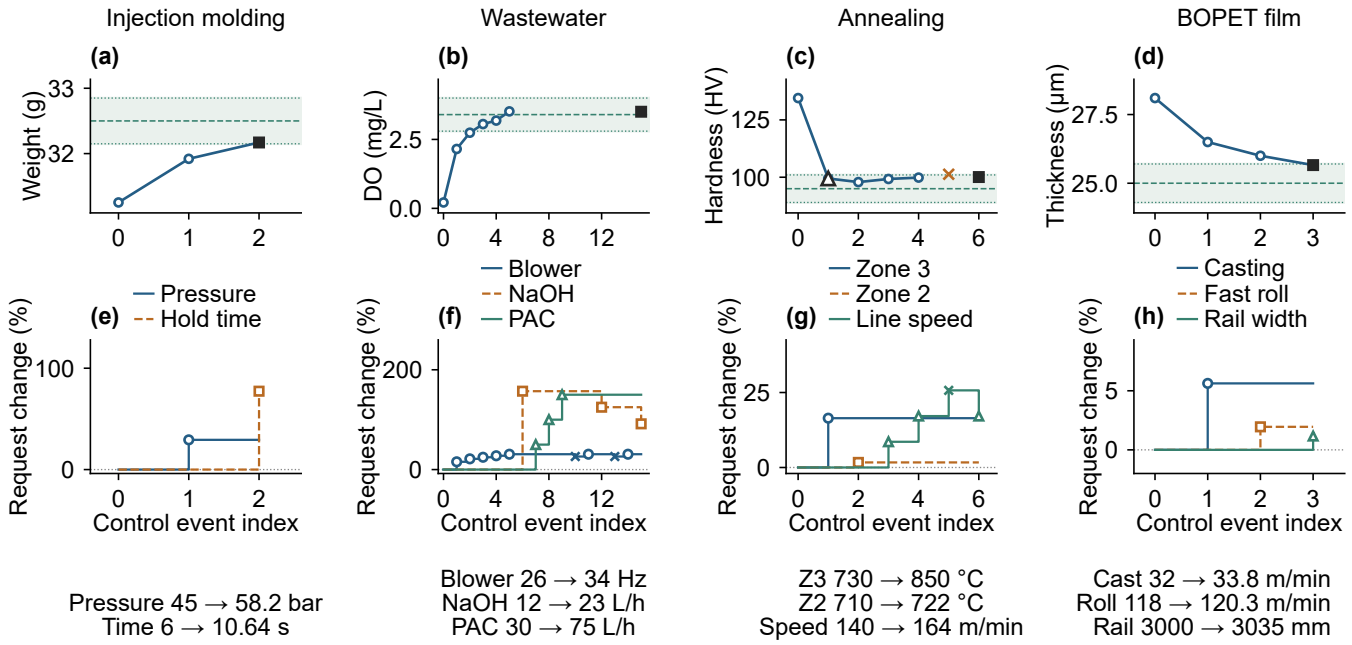


Fig. 5. Historical scripted missions (seed 42, one trajectory per process). (a–d) Weight, dissolved oxygen (DO), hardness, and film thickness with their target windows. (e–h) Requested controls as percentage change from native baselines: injection pressure 45 bar/time 6 s; wastewater blower 26 Hz/NaOH 12 L/h/PAC 30 L/h; annealing zone 3 730°C/zone 2 710°C/speed 140 m/min; BOPET casting 32 m/min/fast roll 118 m/min/rail 3000 mm. Lines connect event indices, not physical time; unchanged requested values carry forward. Crosses mark reverted candidates and squares distinct terminal observations. DO excludes pH/TP events; annealing's first step lacks an optimization record.

Injection molding used two changes: hold pressure $45 \rightarrow 58.2$ bar and hold time $6 \rightarrow 10.64$ s. Part weight rose from 31.25 to 32.17 g for a 32.5 ± 0.35 g window, only 0.02 g above its lower limit. Biaxially oriented polyethylene terephthalate (BOPET) changed casting speed $32 \rightarrow 33.8$ m/min, fast-roll speed $118 \rightarrow 120.3$ m/min, and rail width $3000 \rightarrow 3035$ mm. Thickness fell from 28.10 to 25.66 μm for a 25.0 ± 0.7 μm target, 0.04 μm inside the upper limit. Its three changes each have a recorded keep verdict and journal identifier.

Wastewater treatment required 15 writes across several nodes. Dissolved oxygen rose from 0.21 to 3.50 mg/L, within 3.4 ± 0.6 mg/L. Some admissible blower proposals reduced compliance margin and were reverted. Terminal guards remained within their declared limits, although pH moved toward its lower bound (6.59 to 6.38; limit 6.0). Dissolved oxygen, total phosphorus (TP), and pH are different quantities and are not concatenated into one output trajectory.

Annealing combined furnace-temperature changes with line-speed increases from 140 to 164 m/min. The recorded Vickers hardness (HV) trajectory moved from 134.52 to 99.84 HV; the separate terminal measurement was 100.08 HV, within 95 ± 6 HV. A 176 m/min attempt produced 101.3 HV and was reverted. The first temperature step has no optimization record identifier in the archive, which bounds record-level traceability. These historical missions document process behavior under a shared scripted task scaffold and retain the distinction between a candidate command, a reverted candidate, and a terminal observation. They do not evaluate current profile or per-agent binding variation across scenes.

C. Heterogeneous Protocol Integration and Route Checks (Q1)

The suite acquired 55 samples over five protocol families and issued policy-checked writes over four; the RTU-over-TCP line was acquisition-only. Archived upper-median write latencies ranged from 19.3–47.8 ms, with observed maxima 20.1–248.9 ms. The four writable lines accepted legal writes and rejected 24/24 out-of-window probes. A configuration-only second preset added 9/9 rejected probes on three further writable lines. These results exercise the same resolver against different signal maps.

Table II isolates admission from confirmation in a mock-driver API fixture. Each arm replayed six attack classes in three deterministic repetitions (18 attacks per arm); separate boundary, legal, and timed probes are excluded from that denominator. Removing the soft-window interlock allowed six soft-window probes to execute, while the hard-range probes remained refused. Archived outcomes and journal records link the refusals and executed commands to their configured arms. Removing readback did not alter admission. Confirmation can therefore check an execution outcome, but cannot replace a pre-write gate. A frozen-measurement drill further separated command and process evidence: the setpoint and readback agreed, yet the monitoring quantity remained outside its window and triggered backstop rollback after 130.2 s. A manual-binding drill suspended execution until harness-issued approval (461 ms mediation span).

D. Live-Engine Tasks and Deterministic Controller (Q2–Q3)

The archived team workflow linked a parent task, a bound worker, a command, a verdict, and its journal. On the single-

TABLE II
MOCK-DRIVER API ABLATION: SIX ATTACK CLASSES, THREE DETERMINISTIC PROPERTY-TEST REPETITIONS PER ARM (18 ATTACKS, EXCLUDING SEPARATE BOUNDARY PROBES). FOUR HARD-RANGE CLASSES ARE REFUSED IN EVERY ARM; TWO SOFT-WINDOW CLASSES CHANGE OUTCOME.

Arm	Soft window	Readback	Refused	Executed
full	✓	✓	18/18	0
no-readback	✓	—	18/18	0
no-interlock	—	✓	12/18	6
ungated	—	—	12/18	6

node cast-film mission, the policy changed the heater from 200 to 204.704 °C; the observation settled at 204.70 °C within the declared target. The task reached `COMPLETED`. The BOPET mission expanded this workflow to nine devices and 49 signals, with three manipulated nodes (Section V-B).

Two live-engine tasks used the same `omp` harness, Zhipu Coding Plan provider, and GLM-5.3-Flash model on a scratch line; they are execution traces, not a strategy comparison or a cross-harness portability test. The archive records a bounded write, a goal-driven descent to 51.7 μm for a $52.0 \pm 0.8\,\mu$m target, and a recipe save. The original contemporaneous omp logs and checked run outcomes trace proposal, policy-checked action, observations, and retained recipe. The reported 51.7 μm is the run-summary terminal value; the log's 52.02 μm window mean is a different aggregation.

A deterministic controller used the dimensionless cast-film objective

$$J = 55J_{\text{thick}} + 25J_{\text{quality}} + 8J_{\text{energy}} + 7J_{\text{throughput}}, \tag{2}$$

whose components score thickness conformance, product quality, and control-based energy and throughput proxies. The latter two terms use screw speed and line speed, rather than independently metered energy and production output. This deterministic-controller experiment is not an evaluation of AgentWorkShop's optional model service. Table III summarizes its finite candidate grid and selection rule; component formulas and replay details are provided in `REPRODUCIBILITY.md`.

Across seeds 42–44, this single controller made two policy-checked writes per seed, without rejection. It raised J from approximately 67.2 to $J/J^* = 0.965$–0.972, with $J^* = 89.894$ from the admissible setpoint grid. This reference normalization is restricted to the finite search space. The result links the controller's native objective to admissible commands and endpoint evidence, exercising a conventional strategy through the same execution services.

E. Deployment Composition and Confirmation Semantics (Q5)

The deployment instance contained 34 provisioned lines, 111–113 channels, 67 online DAQ nodes, 77 write nodes, 52 recipes, and 385 bindings, with 17 lines running. These objects coexisted on one simulator-backed host. Nine pipeline executions passed 782 checks (Table IV).

The injection benchmark followed observation, proposal, pacing gate, approval, batch dispatch, and recipe/readback/mirror comparison. The three sources agreed at the endpoint's recording resolution (maximum deviation 0.0 bar; tolerances 0.05 bar for device readback and 0.51 bar for the bucketed mirror). This verifies setpoint consistency rather than process convergence. In S4, the initial proposal was cadence-refused; the reproposal followed 69.014 s later, and observation-to-verification at 65 bar took 96.105 s. The archived verdict remained `converged=false`. S5 began 96.129 s after the S4 observation, applied a 40-to-40.5°C setpoint, refused a further write, waited 66.046 s, and restored 40°C. These are server/harness event spans, not process-convergence evidence. Across the campaign, approval-to-dispatch spans were 6–9 ms.

Seven fault tests exercised receipt grading. A mock acceptance required a reread; a broker outage failed one parameter while its healthy sibling verified; and a publish-only endpoint remained unverified after three attempts despite returning a broker receipt. The suite also tested readback-based promotion, confirmation of manual-to-automatic mode changes, and approval before manual-mode recipe dispatch. MES application-receipt semantics were covered by route tests; the campaign itself used MES only in read mode. These checks show that receipt grades followed the available endpoint evidence under the injected conditions.

F. Operational Recipe Workflow Case Study (Q6)

The October 18 recipe archive (R6 in the accompanying reproducibility notes) pairs recipe-path controls, receipts, and process observations. The replay entry is `scripts/e2e-injection-recipe-closedloop.mjs`. R6 lacks an exact source commit, seed, and per-sensor timestamp series, so it is reported as an archived trajectory rather than a fully specified replay. The setup used OPC UA setpoints and HTTP quality acquisition against the injection simulator. A live agent separately diagnosed insufficient packing, but the archived intervention sequence and approvals were scripted.

Starting from 45 bar with a fixed 6-s hold time, the script proposed 5-bar pressure increments and stopped when weight was in $[32.15, 32.85]$ g, flash was at most 0.4%, and sink mark was at most 1.5%, or after six proposals. Each `recipe_trial` was approved by the harness, followed by a 65-s wait, and checked against the platform control node and DAQ. The successful run used four proposals; its overshoot compensation and hold-time fallback were not needed. Table V records the archived parameter and action sequence.

The successful harness set `AW_TRIAL_INTERVAL_MS=0`, disabling per-line trial cadence, and used `emergency=true` to bypass the separate recipe-operation interval; approval, admission, and command confirmation remained active. These snapshots are platform-node values, not a physical PLC trace. Melt temperature was missing, so the pass covers only the three observed quality quantities. The update rationale contains different quality figures without a linked raw sample; the table uses the archived final snapshot.

The freshness check used the newest timestamp among available sensors, not each sensor's individual timestamp. The additional archive is not an independent replication. A separate

TABLE III
FINITE-GRID SEARCH PROTOCOL FOR THE DETERMINISTIC CAST-FILM CONTROLLER IN Q3. THE REFERENCE SCORE IS RESTRICTED TO THIS GRID AND IS NOT A GLOBAL CONTINUOUS OPTIMUM.

Search element	Candidate set or rule	Role in the search
Objective	Eq. (2); weights 55/25/8/7 for thickness, quality, energy proxy, and throughput proxy	Rank eligible steady states
Eligibility	Melt temperature 195–225 °C; melt pressure $\leq$ 22 MPa	Discard ineligible states
Equal zone set-point	195–225 °C in 2.5 °C steps	Search variable
Screw speed	50–200 rpm in 5-rpm steps	Search variable
Line speed	20–120 m/min in 2.5-m/min steps	Search variable
Die gap	Fixed at 1 mm	Held constant
Selection	Evaluate modeled steady state and retain the highest eligible score; $J^* = 89.894$	Finite-grid reference

TABLE IV
DEPLOYMENT-SUITE ASSERTIONS ACROSS NINE SEQUENTIAL EXECUTIONS; CHECKS ARE DETERMINISTIC ASSERTIONS, NOT INDEPENDENT PLANT SAMPLES.

Pass	Test surface	Runs	Checks
1	baseline S0–S7	3	237
2	+ grading and master switch	3	273
3	hardened regression surface	3	272
Total	zero failed checks	9	782

later check started at 65 bar, 32.29 g, and 1.385% sink mark; one 70-bar trial ended at 32.55 g and 0.976%. It is a new initial state and is not appended to the plotted trace.

October 23 boundary checks distinguish historical replay from current operation. The public interface refused all eight direct-node attempts (zero writes). A separate recipe check verified one 45-to-50-bar trial, then refused requests at 66 and 132 s under the default 300-s cadence. Restoration failed: pressure remained at 50 bar rather than the requested 45-bar baseline, and only 4/8 harness assertions passed. Final weight was 31.46 g; no recipe update was saved. These failed checks are not repetitions of the accelerated October 18 trajectory; their full outcomes are retained in the reproducibility notes.

VI. DISCUSSION

R1 contains four seed-42 process trajectories under a shared scripted scaffold; these historical scene configurations reached their declared terminal checks. Q1 covers acquisition over five protocol families and writes over four mappings, while Q5 checks deployment behavior in nine sequential suite runs. Together, these archives support integration of the tested interfaces and workflows. They do not show that current channel profiles and per-agent bindings preserve tool availability and task outcomes across scenes: Q4 used a historical direct-node path, and the independent profile checks used temporary storage and mock I/O.

Q6 is one inspectable recipe trajectory: its harness disabled the line cadence and used emergency trials to bypass the separate recipe-operation interval. The accepted definition was saved but not applied. A default-paced attempt missed its target. Separately, an October 23 direct-node replay refused all eight historical writes, and a default-paced recipe check failed restoration, leaving pressure at 50 bar rather than the requested 45 bar (four of eight assertions passed). These records do not establish default-paced recipe completion or recovery readiness.

AgentWorkShop composes agent collaboration, scene configuration, production analysis, semantic device access, and route-specific execution records. Its interfaces let an operator inspect a proposal, endpoint evidence, and process observation separately. The evaluation does not establish operator effectiveness: approval records do not measure comprehension or complacency [24]–[26]. Likewise, a restricted tool surface does not remove deployment security requirements [19], [27], [28], and the framework does not replace independently engineered PLC or safety-instrumented protection under the IEC 61511 process-safety lifecycle [29].

Next, a versioned build should vary profiles and bindings across matched scenes, measure tool availability and task outcomes, and test recipe recovery at default pacing against declared baselines. Physical-plant evaluation is needed before claiming transfer beyond repository simulators; Tennessee Eastman is one additional setting [30]. Model-assisted control response remains unevaluated.

VII. CONCLUSION

AgentWorkShop provides a modular integration path linking collaborative harnesses, production context, analysis and modeling services, semantic industrial I/O, and node and recipe workflows. It keeps task and context resolution, dispatch, command confirmation, process observation, and route-specific evidence within explicit module interfaces. Historical records show terminal-check success in four simulator scenes; separate archives exercise protocol, deployment, live-engine, controller, and recipe functions. The evidence supports the tested interfaces and routes, but not current cross-scene profile/binding adaptation, cross-harness portability, default-paced recipe recovery, physical transfer, or comparative control benefit. Matched cross-scene evaluation is the next step; plant-level effectiveness remains unestablished.

REFERENCES

[1] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[2] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

TABLE V
RECORDED Q6 INJECTION-RECIPE ACTIONS AND POST-WAIT OBSERVATIONS ($n = 1$ SIMULATOR TRAJECTORY). HOLD TIME REMAINED 6 S; EACH TRIAL USED A HARNESS-SUPPLIED APPROVAL, A 65-S WAIT, AND A 1/1 PARAMETER-VERIFICATION RECEIPT.

Stage	Pressure (bar)	Weight (g)	Flash (%)	Sink (%)	Recorded action and outcome
Baseline	45	31.28	0	1.977	No command; initial observation
Trial 1	45→50	31.49	0.014	1.956	Approved; 1/1 verified; continue
Trial 2	50→55	31.74	0.031	1.760	Approved; 1/1 verified; continue
Trial 3	55→60	32.05	0	1.534	Weight below 32.15-g minimum and sink above 1.50%; continue
Trial 4	60→65	32.30	0.019	1.277	Three observed guards pass; stop
Save	65	—	—	—	`recipe_update` saved version 2; no `recipe_apply` is archived

[3] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[4] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[12] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[13] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[14] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[15] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[16] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[17] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[18] U.S. Food and Drug Administration, "Part 11, electronic records; electronic signatures—scope and application," Guidance for Industry, 2003.

[19] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[20] IEEE, "IEEE std 1815-2012: Standard for electric power systems communications—distributed network protocol (DNP3)," Standard, 2012.

[21] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[22] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017, doi: 10.1109/TAC.2016.2638961.

[23] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001, doi: 10.1109/MS.2001.936213.

[24] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[25] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[26] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[28] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[29] IEC, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.

[30] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993, doi: 10.1016/0098-1354(93)80018-I.